ТОО ”Колледж Хекслет”

Лабораторная работа №7

Связанные данные и запросы в Prisma

Выполнено студентом:

Дубинин Кирилл

Практическое задание

Раздел RO4_3. Связанные данные и запросы в Prisma.

Цель: Научиться получать связанные данные Product, Category, Order и OrderItem через Prisma Client, формировать вложенные JSON-ответы и фильтровать записи по связанным моделям.

Условие: RO4_3/lesson7/ЛР-7.pdf

Требования задания

• Product API возвращает связанную Category.

• Для ограничения вложенных полей используется select.

• Order API возвращает связанного User.

• Order API возвращает массив связанных OrderItem.

• Каждый OrderItem содержит данные связанного Product.

• В ответ заказа добавляется вычисляемое поле total.

• Поиск заказов по товару использует relation filter some.

• Prisma-запросы находятся в Service, а не в Controller или Route.

• Отсутствующие записи корректно приводят к HTTP 404.

• Данные фактически читаются из PostgreSQL.

Ход выполнения

Product загружается вместе с Category; вложенный select ограничивает поля категории. Заказ содержит безопасные поля User, позиции OrderItem и названия Product. total вычисляется из OrderItem.price, а не текущей цены каталога. Relation filter some находит заказы с указанным productId.

Проект: RO4_3/lesson7

Запуск: npm install; npm start. Для PostgreSQL параметры подключения задаются в .env по образцу .env.example. Порядок применения SQL и миграций приведён в README.md проекта.

services/productService.js

services/productService.js

```
import { prisma } from '../config/prisma.js';
export async function getAllProducts(query = {}) {
    const where = {};
    if (query.category) where.category = { name: query.category };
    if (query.inStock !== undefined) where.inStock = query.inStock === 'true';
    if (query.minPrice !== undefined || query.maxPrice !== undefined) {
        where.price = {};
        if (query.minPrice !== undefined) where.price.gte = Number(query.minPrice);
        if (query.maxPrice !== undefined) where.price.lte = Number(query.maxPrice);
    }
    return prisma.product.findMany({ where, orderBy: { id: "asc" }, include: { category:
 { select: { id: true, name: true } } } });
}
export function findProductById(id) { return prisma.product.findUnique({ where: { id },
include: { category: { select: { id: true, name: true } } } }); }
function productData(input) {
    return Object.fromEntries(['name', 'price', 'categoryId', 'inStock'].filter(key =>
key in input).map(key => [key, input[key]]));
}
export function addProduct(data) { return prisma.product.create({ data:
productData(data) }); }
export async function updateProductById(id, changes) {
    try { return await prisma.product.update({ where: { id }, data: productData(changes)
 }); }
    catch (error) { if (error.code === 'P2025') return null; throw error; }
}
export async function deleteProductById(id) {
    try { return await prisma.product.delete({ where: { id } }); }
    catch (error) { if (error.code === 'P2025') return null; throw error; }
}
```

Листинг 1. services/productService.js

services/orderService.js

```
services/orderService.js
```

```
import { prisma } from '../config/prisma.js';
import { AppError } from '../errors/AppError.js';
const relations = {
    user: { select: { id: true, name: true, email: true } },
    items: { include: { product: { select: { id: true, name: true } } } }
};
export function findOrderById(id) {
    return prisma.order.findUnique({ where: { id }, include: relations });
}
export async function getOrderDetails(id) {
    const order = await findOrderById(id);
    if (!order) return null;
    return { ...order, total: order.items.reduce((sum, item) => sum + item.price *
item.quantity, 0) };
}
export function findOrdersByProduct(productId) {
    return prisma.order.findMany({ where: { items: { some: { productId } } }, include:
relations, orderBy: { id: 'asc' } });
}
export function getAllOrders() {
    return prisma.order.findMany({ include: relations, orderBy: { id: 'asc' } });
}
export async function createOrder(userId, items) {
    return prisma.$transaction(async tx => {
        const user = await tx.user.findUnique({ where: { id: userId } });
        if (!user) throw new AppError('User not found', 404);
        const order = await tx.order.create({ data: { userId, status: 'new' } });
        for (const item of items) {
            const product = await tx.product.findUnique({ where: { id: item.productId }
});
            if (!product || !product.inStock) throw new AppError(`Product
${item.productId} is unavailable`, 409);
            await tx.orderItem.create({ data: { orderId: order.id, productId:
product.id, quantity: item.quantity, price: product.price } });
        }
```

Листинг 2. services/orderService.js (часть 1)

```
services/orderService.js
```

```
        const result = await tx.order.findUnique({ where: { id: order.id }, include:
relations });
        return { ...result, total: result.items.reduce((sum, item) => sum + item.price *
 item.quantity, 0) };
    });
}
```

Листинг 3. services/orderService.js (часть 2)

controllers/orderController.js

controllers/orderController.js

```
import * as service from '../services/orderService.js';
import { AppError } from '../errors/AppError.js';
export async function getOrderById(req, res, next) {
    try {
        const order = await service.getOrderDetails(Number(req.params.id));
        if (!order) return next(new AppError('Order not found', 404));
        res.json(order);
    } catch (error) { next(error); }
}
export async function getOrders(req, res, next) {
    try {
        if (req.query.productId !== undefined &&
(!Number.isSafeInteger(Number(req.query.productId)) || Number(req.query.productId) <=
0)) throw new AppError('Invalid productId', 400);
        res.json(req.query.productId ? await
service.findOrdersByProduct(Number(req.query.productId)) : await
service.getAllOrders());
    } catch (error) { next(error); }
}
export async function createOrderController(req, res, next) {
    try { res.status(201).json(await service.createOrder(req.body.userId,
req.body.items)); }
    catch (error) { next(error); }
}
```

Листинг 4. controllers/orderController.js

Проверка результата

Проверено: 05.11.2026 13:38 (Алматы). Node.js v26.12.1.

Приведены фактические результаты проверки. Полный журнал: verification.json.

• Реальный PostgreSQL

• HTTP endpoints

• CRUD сохраняется в PostgreSQL

• UNIQUE и FOREIGN KEY; категории перенесены без потери товаров

• Связи 1:1, 1:N, M:N; CHECK(quantity > 0); составной ключ

• include/select, relation filter some, историческая сумма заказа

• Prisma Studio открыт в Chromium; таблица products содержит Keyboard

HTTP-запросы

GET /api/products → 200

GET /api/products/1 → 200

GET /api/products/999999 → 404

GET /api/products/abc → 400

GET /api/products?category=Accessories&inStock=true&minPrice=10000&maxPrice=50000 → 200

GET /api/products?minPrice=abc → 400

GET /api/products?inStock=maybe → 400

GET /api/products?minPrice=500&maxPrice=100 → 400

POST /api/products → 403

POST /api/products → 400

POST /api/products → 201

PATCH /api/products/6 → 200

DELETE /api/products/6 → 200

GET /api/products/6 → 404

POST /api/products → 409

GET /api/orders/1 → 200

GET /api/orders?productId=1 → 200

GET /api/orders/999999 → 404

GET /api/unknown → 404

Примеры ответов

POST /api/products → 201


```
{
  "id": 6,
  "name": "Test SSD",
  "price": 65000,
  "categoryId": 1,
  "inStock": true
}
```

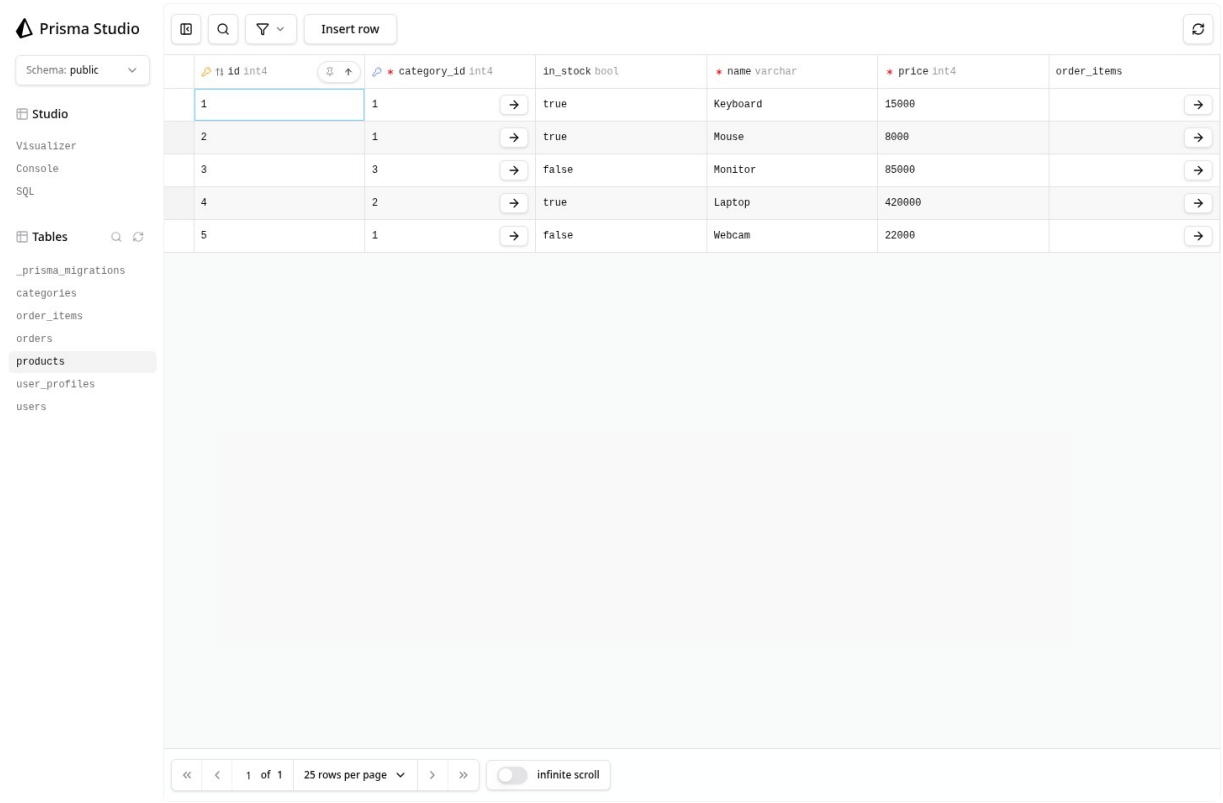

PATCH /api/products/6 → 200


```
{
  "id": 6,
  "name": "Test SSD",
  "price": 42000,
  "categoryId": 1,
  "inStock": false
}
```


DELETE /api/products/6 → 200


```
{
  "id": 6,
  "name": "Test SSD",
  "price": 42000,
  "categoryId": 1,
  "inStock": false
}
```

Проверка данных в Prisma Studio



Prisma Studio: таблицы и данные проверенной учебной базы.

Результат

Проверены обязательные операции и указанные ошибочные сценарии; результаты сохранены вместе с исходным кодом.